

一、QWEN2-0.5B 模型微调

服务器选择(7B 以下的 24G 的服务器均可以):

建议阿里云的 DSW ecs.gn7i-c8g1.2xlarge 即可

基础镜像选择: modelscope:1.14.0-pytorch2.1.2tensorflow2.14.0-gpu-py310-cu121-ubuntu22.04

1. Llama-Factory 安装

▼ 安装环境

纯文本

```
# 参考: https://github.com/hiyouga/LLaMA-Factory/blob/main/README_zh.md
# NOTE: llamafactory-cli这个命令执行的时候, 当前文件夹最好是LLaMA-Factory

git clone https://github.com/hiyouga/LLaMA-Factory.git
cd LLaMA-Factory
git tag -l
git fetch --tags
git checkout v0.9.2
pip install -e ".[torch,metrics]"

pip uninstall -y vllm
pip install transformers==4.44.2

# 如果出现gradio相关异常, 那么升级一下版本
pip install gradio==5.45.0 gradio-client==1.13.0

llamafactory-cli version
-----
| Welcome to LLaMA Factory, version 0.9.2          |
|                                                  |
| Project page: https://github.com/hiyouga/LLaMA-Factory |
|-----|

##### 修改部分内容
# 修改 LLaMA-Factory/src/llamafactory/extras/misc.py 中的方法 use_modelscope 中,
# 使其默认采用modelscope下载模型
# 也可以通过给定外部环境变量的方式来实现 export USE_MODELSCOPE_HUB=1 ----> 当前窗口生效
def use_modelscope() -> bool:
    # 默认采用modelscope
    return is_env_enabled("USE_MODELSCOPE_HUB", '1')
# 可以在需要的地方添加一些日志, 方便大家理解代码流程
# PS: 如果采用modelscope进行模型下载, 有可能存在代码的bug,
#### 需要修改modelscope的源码: 见当前页面最后
##### 修改部分内容

llamafactory-cli webui
```

=====

点击启动时候显示的url日志

```
root@dsw-930292-5c586966cc-rkqvf:/mnt/workspace/nlp0916/LLaMA-Factory# llamafactory
-cli webui
```

```
[2025-09-18 20:28:45,973] [INFO] [real_accelerator.py:161:get_accelerator] Setting
ds_accelerator to cuda (auto detect)
```

```
* Running on local URL:  http://0.0.0.0:7860
```

To create a public link, set `share=True` in `launch()`.

=====

The screenshot shows the LLaMA-Factory web interface with the following configuration:

- 语言:** zh
- 模型名称:** Qwen2-0.5B
- 模型路径:** Qwen/Qwen2-0.5B
- 微调方法:** lora
- 检查点路径:** (empty)
- 量化等级:** none
- 量化方法:** bitsandbytes
- 对话模板:** default
- RoPE 插值方法:** none
- 加速方式:** auto

Below these settings are tabs for **Train**, **Evaluate & Predict**, **Chat**, and **Export**. The **Train** tab is active, showing:

- 训练阶段:** Supervised Fine-Tun
- 数据路径:** data
- 数据集:** (empty)
- 学习率:** 5e-5
- 训练轮数:** 3.0
- 最大梯度范数:** 1.0
- 最大样本数:** 100000
- 计算类型:** bf16
- 截断长度:** 2048
- 批处理大小:** 2
- 梯度累积:** 8
- 验证集比例:** 0
- 学习率调节器:** cosine

2. 测试一下 Qwen0.5B 模型原始的效果

Python

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_dir = "/mnt/workspace/.cache/modelscope/Qwen/Qwen2-0.5B"
tokenizer = AutoTokenizer.from_pretrained(model_dir)
model = AutoModelForCausalLM.from_pretrained(model_dir)

print(type(tokenizer))
print(type(model))
model

text = "你是谁"
input_ids = tokenizer(text, return_tensors = "pt")['input_ids']
# 生成方式采用贪心的方法
generate_output = model.generate(input_ids, max_new_tokens=50, do_sample=False)
```

```

# 将input_ids部分剔除掉
response_ids = generate_output[:, len(input_ids[0]):]
response = tokenizer.batch_decode(
    response_ids, skip_special_tokens=True, clean_up_tokenization_spaces=True
)
print(response)
#### 输出结果为
# ['你就是我们班的“小博士”——王佳怡。你有一双明亮的大眼睛，一张能说会道的嘴巴，你是一个聪明、活泼、可爱的小女孩。你上课认真听讲，积极发言，作业']
####

text = "Human: 你是谁\nAssistant:"
input_ids = tokenizer(text, return_tensors = "pt")['input_ids']
generate_output = model.generate(input_ids, max_new_tokens=50, do_sample=False)
response_ids = generate_output[:, len(input_ids[0]):]
response = tokenizer.batch_decode(
    response_ids, skip_special_tokens=True, clean_up_tokenization_spaces=True
)
response
#### 输出结果为
# [' 1、你是一个聪明的孩子，你有很强的求知欲，你有很强的观察力，你有很强的想象力，你有很强的表达能力，你有很强的动手能力，你有很强的思维能力，']
####

del tokenizer
del model

```

3. 微调模型

▼ 构造训练数据，并放到 Llama-Factory/data 文件夹中

纯文本

1. 构造微调模型的训练数据 --> 按照对应的格式

单轮对话的格式 也是文本生成的格式

```

{
    "instruction": "hello",
    "input": "",
    "output": "Hello! I am MINI-GPT, an AI assistant developed by 小明. How can I assist you today?"
}

```

多轮对话的格式

```

{
    "conversations": [
        {
            "from": "human",
            "value": "假设你有一个需要随机数的Java程序，范围在0到10之间。你可以使用什么代码片段来生成这样的数字？\n\n下面是一个示例代码片段，它使用Math.random()方法来生成一个在0到1之间的随机数，然后缩放并四舍五入结果，得到一个在0到10之间的随机数：\n\n```\njava\nint randomNumber = (int) (Math.random() * 11);\n```\n\n在Java中是否有更有效的方法来生成随机数？你会如何修改代码以生成具有不同范围或分布的随机数？"
        },
        {
            "from": "gpt",
            "value": "您提供的代码片段是Java中生成随机数的一种常见且高效的方法。然而，如果您需要对随

```

机数生成有更多的控制，您可以使用java.util.Random类，它提供了更多生成随机数的选项。
要生成一个在不同范围内的随机数，您可以修改代码以调整缩放因子。例如，要生成一个在1到10之间的随机数，您可以使用以下代码：
`int randomNumber = (int) (Math.random() * 10) + 1;`
要生成具有不同分布的随机数，您可以使用各种技术，如正态分布、指数分布或自定义分布。生成正态分布的一种方法是使用Box-Mu
ller变换。这里有一个代码示例，它生成具有正态分布的随机数：
`Random random = new Random();
double u1 = random.nextDouble();
double u2 = random.nextDouble();
double z = Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
double mean = 5;
double stdDev = 2;
int randomNumber = (int) (z * stdDev + mean);`
这段代码生成一个具有均值5和标准差2的正态分布随机数。您可以调整均值和标准差来生成具有不同分布的随机数。

```
    },  
    {  
      "from": "human",  
      "value": "你能解释一下算法分析中的大O记法概念吗？"  
    },  
    {  
      "from": "gpt",  
      "value": "是的，大O表示法是一种描述算法性能随输入数据大小增加的方法。它用于估计算法在最坏情况下的时间复杂度或空间复杂度。大O中的“O”代表“数量级”，这意味着随着输入大小的增加，算法所需的操作数  
以一定的速率增长。  
例如，如果我们有一个需要n步才能解决大小为n的问题的算法，那么时间复杂度可以表示为O(n)，这意味着操作数随着输入大小线性增长。如果我们有一个需要n^2步才能解决大小为n的问题的算法，那么时间复杂度可以表示为O(n^2)，这意味着操作数随着输入大小二次方增长。  
还有其他几种大O表示法，如O(log n)、O(n log n)、O(2^n)等，它们代表了随着输入大小增加的不同操作数增长速率。通过分析算法的大O表示法，我们可以比较不同算法的性能，并选择给定问题最有效的算法。  
您是否有关于大O表示法或算法分析的特定问题？"  
    }  
  ],  
  "tools": "[]"  
}
```

纯文本

2. 修改Llama-Factory/data/dataset_info.json文件内容

添加内容

```
"new-identity": {  
  "file_name": "new_identity.json"  
},
```

添加内容

3. 重启web ui

```
llamafactory-cli webui
```

4. 在页面上选择相应的配置信息，进行微调操作(train这个标签)

配置文件保存到 Llama-Factory/config 这个文件夹中

微调好的模型参数保存到 Llama-Factory/save 这个文件夹中

5. 在训练好后，选择export标签，可以将微调好的模型导出使用

Checkpoint path 选择train训练好后模型的保存路径(一般刷新页面后，点击可以自动弹出)

Export dir 在保存输出框中，加入保存文件夹路径 export/qwen/Qwen2-0.5B-20241218210658

点击Export即可

6. 和export一样选择checkpoint path后，可以在chat标签页面中选择load model进行对话测试

4. 测试微调后的模型效果

Python

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_dir = "xxxx/export/qwen/Qwen2-0.5B"
tokenizer = AutoTokenizer.from_pretrained(model_dir)
model = AutoModelForCausalLM.from_pretrained(model_dir)

print(type(tokenizer))
print(type(model))
model

text = "你是谁"
input_ids = tokenizer(text, return_tensors = "pt")['input_ids']
# 生成方式采用贪心的方法
generate_output = model.generate(input_ids, max_new_tokens=50, do_sample=False)
# 将input_ids部分剔除掉
response_ids = generate_output[:, len(input_ids[0]):]
response = tokenizer.batch_decode(
    response_ids, skip_special_tokens=True, clean_up_tokenization_spaces=True
)
print(response)
#### 输出结果为
# ['? 我是 MINI-GPT, 一个由 小明 开发的人工智能助手。']
####

text = "Human: 你是谁\nAssistant:"
input_ids = tokenizer(text, return_tensors = "pt")['input_ids']
generate_output = model.generate(input_ids, max_new_tokens=50, do_sample=False)
response_ids = generate_output[:, len(input_ids[0]):]
response = tokenizer.batch_decode(
    response_ids, skip_special_tokens=True, clean_up_tokenization_spaces=True
)
response
#### 输出结果为
# ['您好, 我是 MINI-GPT, 一个由 小明 开发的人工智能助手, 很高兴认识您。']
####

del tokenizer
del model
```

扩展:



<https://github.com/liguodongiot/llm-action>



<https://github.com/datawhalechina/llm-cookbook>



<https://github.com/datawhalechina/hugging-llm>

异常:

1. 模型下载异常

解决方案: 修改 modelscope/hub/file_download.py 中的 http_get_file 方法中的长度判断的代码, 进行注释即可。

```
File ~/opt/conda/lib/python3.10/site-packages/gradio/utils.py, line 671, in async_iteration
    return await iterator.__anext__()
File ~/opt/conda/lib/python3.10/site-packages/gradio/utils.py, line 664, in __anext__
    return await anyio.to_thread.run_sync(
File ~/opt/conda/lib/python3.10/site-packages/anyio/to_thread.py, line 56, in run_sync
    return await get_async_backend().run_sync_in_worker_thread(
File ~/opt/conda/lib/python3.10/site-packages/anyio/_backends/_asyncio.py, line 2134, in run_sync_in_worker_thread
    return await future
File ~/opt/conda/lib/python3.10/site-packages/anyio/_backends/_asyncio.py, line 851, in run
    result = context.run(func, *args)
File ~/opt/conda/lib/python3.10/site-packages/gradio/utils.py, line 647, in run_sync_iterator_async
    return next(iterator)
File ~/opt/conda/lib/python3.10/site-packages/gradio/utils.py, line 809, in gen_wrapper
    response = next(iterator)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/webui/chat.py, line 107, in load_model
    super().__init__(args)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/chat/chat_model.py, line 44, in __init__
    self.engine = BaseEngine = HuggingfaceEngine(model_args, data_args, finetuning_args, generating_args)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/chat/hf_engine.py, line 53, in __init__
    tokenizer_module = load_tokenizer(model_args)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/model/loader.py, line 67, in load_tokenizer
    init_kwargs = _get_init_kwargs(model_args)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/model/loader.py, line 52, in _get_init_kwargs
    model_args.model_name_or_path = try_download_model_from_ms(model_args)
File ~/mnt/workspace/LLaMA-Factory/src/llamafactory/extra/misc.py, line 225, in try_download_model_from_ms
    return snapshot_download(model_args.model_name_or_path, revision=revision, cache_dir=model_args.cache_dir)
File ~/opt/conda/lib/python3.10/site-packages/modelscope/hub/snapshot_download.py, line 156, in snapshot_download
    http_get_file(
File ~/opt/conda/lib/python3.10/site-packages/modelscope/hub/file_download.py, line 345, in http_get_file
    raise FileDownloadError(msg)
modelscope.hub.errors.FileDownloadError: File tokenizer.json download incomplete, content_length: None but the
Keyboard interruption in main thread... closing server. file downloaded length: 7028015, please download again
```

```
cookies=cookies,
timeout=API_FILE_DOWNLOAD_TIMEOUT)
r.raise_for_status()
content_length = r.headers.get('Content-Length')
total = int(
    content_length) if content_length is not None else None
progress = tqdm(
    unit='B',
    unit_scale=True,
    unit_divisor=1024,
    total=total,
    initial=downloaded_size,
    desc='downloading',
)
for chunk in r.iter_content(
    chunk_size=API_FILE_DOWNLOAD_CHUNK_SIZE):
    if chunk: # filter out keep-alive new chunks
        progress.update(len(chunk))
        temp_file.write(chunk)
progress.close()
break
except (Exception) as e: # no matter what happen, we will retry.
    retry = retry.increment('GET', url, error=e)
    retry.sleep()

logger.debug('retrying %s in cache at %s', url, local_dir)
downloaded_length = os.path.getsize(temp_file.name)
# if total != downloaded_length:
#     os.remove(temp_file.name)
#     msg = 'File %s download incomplete, content_length: %s but the \
#         file downloaded length: %s, please download again' % (
#         file_name, total, downloaded_length)
#     logger.error(msg)
#     raise FileDownloadError(msg)
os.replace(temp_file.name, os.path.join(local_dir, file_name))
```



345,5 底部